

vr²FHE - Securing FHE from Reaction-Based Key Recovery Attacks

Bhuvnesh Chaturvedi¹  , Anirban Chakraborty² , Ayantika Chatterjee¹ , Debdeep Mukhopadhyay¹ 

¹Indian Institute of Technology Kharagpur, ²Max Planck Institute for Security and Privacy[†]

{bhuvneshchaturvedi2512, ch.anirban00727, cayantika, debdeep.mukhopadhyay}@gmail.com

Abstract—Fully Homomorphic Encryption (FHE) inherently lacks data integrity mechanisms, allowing a malicious server to arbitrarily tamper with the data associated with FHE computations on its end. A series of works named *reaction attacks* further demonstrates that a malicious server can exploit this lack of integrity checks to carry out interactive full-key recovery attacks on state-of-the-art FHE schemes. In this paper, we propose an efficient solution to this problem by utilizing the concept of the Merkle tree. Our solution uses cryptographic hash functions to ensure the integrity of data involved in FHE computations. The efficiency of our solution comes from the lower sizes and ease of computations of the hash values, which subsequently leads to a reduction in both the network and computation overhead. Given that protecting the entire FHE circuit can lead to tremendous network overhead, we further perform scheme-specific optimizations by identifying a small portion of the FHE circuit, protecting which is sufficient to thwart these reaction-based attacks. Finally, we propose a framework that evaluates different FHE schemes based on the overhead that will be incurred when protecting these schemes through our solution. Our framework can be leveraged by application developers to choose an optimal FHE scheme in terms of both performance and overhead.

Keywords—FHE, IND-CVA, Reaction Attack, Full Key Recovery, Application Level Security, Merkle tree.

I. INTRODUCTION

Fully Homomorphic Encryption (FHE) [11] is a class of encryption schemes that supports arbitrary computations directly over encrypted data without decrypting it first. This allows a resource-constrained client to offload its private data to a public cloud server while still using the server’s computational services. The result of such computations remains encrypted and can only be decrypted by the client using its secret key. Thus the server remains oblivious to both the inputs and the output of such computations.

Computation over encrypted data requires the ciphertexts to be malleable. In other words, given a ciphertext c that encrypts a message m , an evaluator should be able to modify c to a different ciphertext c' that encrypts another message m' such that $m' = f(m)$ for some known function f . In FHE, this can be achieved by homomorphically evaluating f over c . Due to the malleable nature of the ciphertexts, FHE in itself cannot ensure data integrity. This implies that an adversary can tamper with the ciphertext c' such that it decrypts to a

message $m'' \neq f(m)$. Due to the lack of integrity checks in place, these modifications remain undetected.

In the general FHE setting, the server¹ is often assumed to be *honest-but-curious*. In other words, the server might try to infer information about the client’s data, but it is expected to follow the protocol specification of the requested FHE computation. However, in real-world settings, the server can be *malicious* and can deviate arbitrarily from the protocol. Such a server can exploit the lack of integrity checks in FHE ciphertexts to leak secret information. These attacks are motivated by the fact that the data associated with the FHE computation is private to the client² and it remains encrypted throughout the duration of the computation, preventing the server from accessing it.

A recent class of works [3], [6], [14], [22], named *reaction attacks*, has shown that a malicious server can tamper with the FHE ciphertexts and observe its effect on the decrypted result by utilizing the client as a verification oracle. These attacks exploit the *reaction* [12], [13] of the client when it finds that the post-decrypted result of the requested computation has failed to satisfy predefined application-level constraints [3], [21], thereby assuming the result to be incorrect. If successful, such attacks can lead to the recovery of the entire secret key, and in turn the entire private data, of the client.

To address this issue, researchers have tried to incorporate integrity checks into the FHE ciphertexts by means of Zero Knowledge Proofs (ZKPs) [1], [21]. However, these solutions suffer from major network and computational overhead when instantiated in practice [7]. The network overhead comes from the necessity of transmitting all the proofs and the associated FHE ciphertexts generated during the homomorphic computation to the client. On the contrary, the computation overhead comes from the fact that the proof generation phase involves non-trivial operations that are compute- and memory-intensive. Combining such heavier solutions with the already slow FHE computations can lead to further inefficiencies.

Our Contributions. In this paper, we propose vr²FHE, an efficient solution to thwart the reaction-based attacks with minimal impact on the efficiency of the overall FHE application. Our proposed method allows a client to verify whether the ciphertexts it received have not been tampered with, thereby ensuring data integrity. Moreover, it additionally allows the client to use the received data to repair the computation result locally if it turns out to be wrong. Our method is based

[†] Work partially done while at IIT Kharagpur

¹The terms cloud and server are used interchangeably throughout the paper.

²We assume the function f to be evaluated is public.

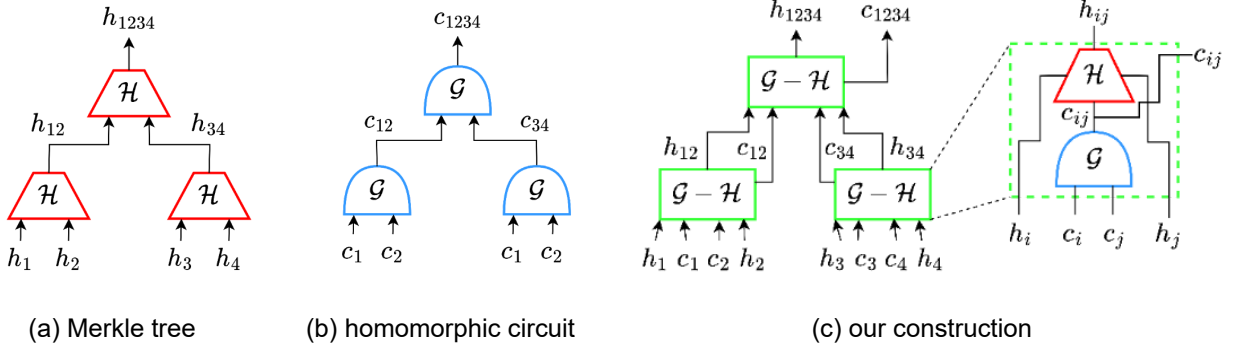


Figure 1: A brief overview of the evaluation of (a) Merkle tree, (b) homomorphic circuit, and (c) our construction. $\mathcal{H}$ = hash function (Sect. II-A2), $\mathcal{G}$ = any homomorphic gate (Sect. II-C), $\mathcal{G} - \mathcal{H}$ = our gate construction (Sect. III-B)

on incorporating the concept of Merkle tree [16] into the FHE circuit (Fig. 1(c)), due to their similar structure and computation direction. These similarities arise from the fact that both the generation of a Merkle tree (Fig. 1(a)) and the evaluation of an FHE circuit (Fig. 1(b)) are done in a layered manner, where the output of layer i becomes the input of layer $i+1$. Moreover, both these computations are done in a bottom-up manner, starting from the nodes that process the inputs.

We further show that our solution does not require protecting the entire FHE circuit, and it suffices to protect a small portion of the circuit. Concretely, we analyze known attacks against the FHE schemes to derive the exact levels of protection that will be sufficient to mitigate reaction-based attacks. These levels vary across different FHE schemes due to the difference in their underlying operation and message space. Therefore, we propose a framework that takes an FHE scheme as input and outputs the number of levels to be protected when a circuit is built using this scheme. Our framework is designed to assist application developers in selecting appropriate FHE schemes while considering both security and performance factors.

Organization. Sect. II introduces the necessary background. Sect. III explains our proposed solution, while Sect. IV elaborates on our optimizations and describes our framework. Sect. V provides the experimental results. Sect. VI details our discussions, and Sect. VII concludes our paper.

II. BACKGROUND

A. Notations and Mathematical Background

1) *Notations:* Boldface letters like $\mathbf{A}$ and $\mathbf{a}$ denote matrices and vectors/polynomials, resp., and $A[i]$ and $a[i]$ denote their i^{th} entry. $\mathbf{a} \cdot \mathbf{s}$ is used to denote a dot product (resp. multiplication) between two vectors (resp. polynomials). $x \leftarrow \chi$ denotes that x has been sampled from χ according to its probability distribution, while $\mathbf{x} \leftarrow \chi$ denotes that each element of $\mathbf{x}$ has been sampled from χ . $\lfloor \cdot \rfloor$, $\lceil \cdot \rceil$ and $[\cdot]_q$, resp., denotes floor, round and mod q operations. $\mathbb{Z}$ denotes a set of integers and $\mathbb{Z}^k$ denotes a vector of dimension k with elements drawn from $\mathbb{Z}$. $c \in C$ denotes that element c belongs to set C . For a positive integer q , $\mathbb{Z}_q$ denotes a set of integers modulo q . $|S|$ and $|s|$ denote the cardinality and the absolute value of a set S and an

element s . $\cdot\|\cdot$ denotes a concatenation operation. For a security parameter λ , $\text{negl}(\lambda)$ represents any negligible function in λ . PPT denotes a probabilistic polynomial time algorithm.

2) *Hash function:* A hash function $\mathcal{H} : \{0, 1\}^* \rightarrow \{0, 1\}^\ell$ is a function that accepts a string of arbitrary length and outputs a string of fixed length ℓ . A cryptographically secure hash function has the following properties:

- pre-image resistance: Given $y = \mathcal{H}(x)$, it is hard to find x .
- second pre-image resistance: Given x and $y = \mathcal{H}(x)$, it is hard to find a different x' such that $\mathcal{H}(x') = y$.
- collision resistance: It is hard to find any two distinct x and x' such that $\mathcal{H}(x) = \mathcal{H}(x')$.

B. (Ring) Learning With Errors problem

The Learning With Errors (LWE) problem [17] is based on adding random error values to each equation in a system of exact equations to turn it into a system of approximate equations. For brevity, let $k > 1$ be an integer and $\mathbf{s} \leftarrow \mathbb{Z}^k$ be a secret vector. An LWE sample is denoted by a tuple $(\mathbf{a}, b) \in \mathbb{Z}_q^{k+1}$, where $\mathbf{a} \leftarrow \mathbb{Z}_q^k$ and $b = \mathbf{a} \cdot \mathbf{s} + e \in \mathbb{Z}_q$. Here $e \leftarrow \chi$ is an error value. In the ring version, denoted RLWE, the operations are performed over rings $R_q = \mathbb{Z}_q[x]/(x^k + 1)$, where $x^k + 1$ is an irreducible polynomial over the rationals, given k is a power of 2 [15]. An RLWE sample is denoted by a tuple $(\mathbf{b}, \mathbf{a}) \in R_q^2$, where $\mathbf{a} \leftarrow R_q$ and $\mathbf{b} = \mathbf{a} \cdot \mathbf{s} + e \in R_q$. Here $e \leftarrow \chi$ is an error polynomial. Both versions of the problem are considered to be quantum-safe.

C. Fully Homomorphic Encryption

In this section, we define FHE in general and provide the implementation-level details of the practical FHE schemes.

Definition 1. An FHE scheme over a message space $\mathcal{M}$ and ciphertext space $\mathcal{C}$ is a tuple of algorithms (FHE.Gen , FHE.Enc , FHE.Eval , FHE.Decrypt), where the algorithms are defined as follows:

1. $\text{FHE.Gen}(1^\lambda)$: On input the security parameter λ , the PPT key generation algorithm outputs a public key pk and a secret key sk .

2. $FHE.Enc(pk, x)$: On input public key pk and a message $x \in \mathcal{M}$, the PPT encryption algorithm outputs a ciphertext $c \in \mathcal{C}$.
3. $FHE.Eval(pk, f, c_1, \dots, c_\ell)$: On input public key pk , circuit f and a set of ciphertexts $c_1, \dots, c_\ell$ corresponding to plaintext messages $x_1, \dots, x_\ell$, the deterministic evaluation algorithm outputs a ciphertext $c_r \in \mathcal{C}$, where $c_r = f(c_1, \dots, c_\ell)$.
4. $FHE.Dec(sk, c)$: On input secret key sk and a ciphertext c , the deterministic decryption algorithm outputs a plaintext message x .

The *exact* FHE schemes satisfy the condition $FHE.Dec(sk, c_r) = f(x_1, \dots, x_\ell)$. On the other hand, the *approximate* FHE satisfies the condition $FHE.Dec(sk, c_r) \approx f(x_1, \dots, x_\ell)$.

We now provide the implementation-level details of practical FHE schemes, which are divided into four stages, namely *keygen*, *encrypt*, *decrypt*, and *evaluate*. The first three stages run on the client side and the last stage runs on the server side.

LWE-based schemes. In LWE-based FHE schemes, the message space is defined over $\{0, 1\}$. In TFHE [5], these bits are encoded as μ and $-\mu$, where $\mu = 2^{29}$, while in FHEW [9], they are encoded as 0 and 128.

1. $LWE.KeyGen(\lambda)$: Upon input a security parameter λ , this algorithm generates public parameters m, k, q, σ . It then generates a public key $\mathbf{pk}$ as a pair $(\mathbf{A}, \mathbf{b})$, where $\mathbf{A} \leftarrow \mathbb{Z}_q^{m \times k}$ is a random matrix and $\mathbf{b} = \mathbf{A} \times \mathbf{sk} + \mathbf{e} \in \mathbb{Z}_q^m$. Here, $\mathbf{sk} \in \mathbb{B}^k$ is the secret key, with $\mathbb{B} \in \{0, 1\}$ for TFHE and $\mathbb{B} \in \{0, \pm 1\}$ for FHEW, and $\mathbf{e} \leftarrow \chi$ is an error vector. It outputs the key pair $(\mathbf{pk}, \mathbf{sk})$.
2. $LWE.Encrypt(\mathbf{pk}, x)$: Upon input a public key $\mathbf{pk}$ and a message $x \in \{0, 1\}$, it chooses a random subset $r \subseteq \mathbf{pk}$, computes an LWE sample $(\mathbf{a}, b')$ as $\sum r$, and output $c = (\mathbf{a}, b) = (\mathbf{a}, b' + x)$.
3. $LWE.Decrypt(\mathbf{sk}, c)$: Upon input a secret key $\mathbf{sk}$ and a ciphertext c , it computes $\phi = b - \mathbf{a} \cdot \mathbf{sk} = x \pm e$ where ϕ denotes the phase of the message x . In TFHE, the sign of ϕ is checked to output 1 if it is positive and 0 if it is negative. In FHEW, the final resultant bit is computed as $x = \lfloor \frac{x \pm e + 64}{128} \rfloor$. The final output is the message x .
4. $LWE.Evaluate(\mathbf{pk}, c_1, c_2)$: Upon input public key $\mathbf{pk}$ and two ciphertexts $c_1 = (\mathbf{a}_1, b_1)$ and $c_2 = (\mathbf{a}_2, b_2)$, it evaluates a homomorphic Boolean gate as follows. For FHEW, it is computed as $\mathbf{a}'_r = 1024 - (\mathbf{a}_1 + \mathbf{a}_2)$ and $b'_r = b_{gc}^F - (b_1 + b_2)$ under modulo-512, where $1024 = \{1024\}^k$. For TFHE, it is defined as $\mathbf{a}'_r = 0 \pm c \cdot (\mathbf{a}_1 \pm \mathbf{a}_2)$ and $b'_r = b_{gc}^T \pm c \cdot (b_1 \pm b_2)$ under modulo- 2^{32} , where $0 = \{0\}^k$. The final output is a ciphertext $c_r = (\mathbf{a}_r, b_r)$.

In practice, the values of b_{gc}^F and b_{gc}^T differ across the different homomorphic gates (barring NOT) defined in FHEW and TFHE schemes. Moreover, the evaluation stage additionally performs a bootstrapping operation using a bootstrapping key $\mathbf{bk}^3$ to reduce the error grown due to the gate computation. In FHEW, a gate evaluation operation transforms the underlying message space to contain a message from 0 to 3. To perform

an inverse transform, modulus switching is performed where 0 and 3 are mapped to 0 while 1 and 2 are mapped to 1.

RLWE-based schemes. In RLWE-based FHE schemes, the plaintext and ciphertext spaces are defined over rings $\mathcal{P} = R_t = \mathbb{Z}_t/(x^k + 1)$ and $\mathcal{C} = R_q^2$, resp. Here $k = 2^d \in \mathbb{Z}$, for a positive integer d , is the ring dimension, while $t \in \mathbb{Z}$ and $q \in \mathbb{Z}$ are the plaintext and ciphertext modulus, resp.

1. $RLWE.KeyGen(\lambda)$: Upon input a security parameter λ , this algorithm generates public parameters p, k, q, σ . It then generates a public key $\mathbf{pk}$ as a pair of rings $(\mathbf{pk}_0, \mathbf{pk}_1) = ([-(\mathbf{a} \cdot \mathbf{sk} \pm p \cdot \mathbf{e})]_q, \mathbf{a})$, where $\mathbf{a} \leftarrow R_q$ is a random polynomial. Here $\mathbf{sk} \in R_2$ with $R_2 \in \{0, \pm 1\}^k$ is the secret key, and $\mathbf{e} \leftarrow \chi$ is a (scaled) error polynomial. Scaling is done by multiplying each coefficient of $\mathbf{e}$ by a factor p , with $p = 1$ for B/FV [10] and $p = t$ for BGV [2]. It outputs the key pair $(\mathbf{pk}, \mathbf{sk})$.
2. $RLWE.Encrypt(\mathbf{pk}, x)$: Upon input a public key $\mathbf{pk}$ and a message $x \in \{0, 1\}$, it samples three polynomials $\mathbf{u} \leftarrow R_2$ and $\mathbf{e}_1, \mathbf{e}_2 \leftarrow \chi$, and encrypts a message x as $(\mathbf{c}_0, \mathbf{c}_1) = (\mathbf{b}, \mathbf{a}) = ([\mathbf{pk}_0 \cdot \mathbf{u} \pm p \cdot \mathbf{e}_1 + x]_q, [\mathbf{pk}_1 \cdot \mathbf{u} \pm p \cdot \mathbf{e}_2]_q)$. Here the message polynomial x is generated by setting its constant coefficient to the message x (in the case of BGV) or $\Delta \cdot x$ (in the case of B/FV), where $\Delta = \lfloor \frac{q}{t} \rfloor$, while setting the rest of the coefficients as 0.
3. $RLWE.Decrypt(\mathbf{sk}, c)$: Upon input a secret key $\mathbf{sk}$ and a ciphertext c , it computes $\phi = \mathbf{b} + \mathbf{a} \cdot \mathbf{s} = \Delta \cdot m \pm p \cdot \mathbf{e}$ where ϕ denotes the phase of the message x . The final output message is then computed as $x = \lfloor \lfloor \frac{\phi}{\Delta} \rfloor \rfloor_t$, where $\Delta = \lfloor \frac{q}{t} \rfloor$ (resp. $\Delta = 1$) for B/FV (resp. BGV).
4. $RLWE.Evaluate(\mathbf{pk}, c_1, c_2)$: Upon input public key $\mathbf{pk}$ and two ciphertexts $c_1 = (\mathbf{c}_1^0, \mathbf{c}_1^1)$ and $c_2 = (\mathbf{c}_2^0, \mathbf{c}_2^1)$, it evaluates a homomorphic addition as $\mathbf{c}_3^0 = [\mathbf{c}_1^0 + \mathbf{c}_2^0]_q$ and $\mathbf{c}_3^1 = [\mathbf{c}_1^1 + \mathbf{c}_2^1]_q$. Moreover, it evaluates a homomorphic multiplication as $\hat{\mathbf{c}}_3^0 = [\mathbf{c}_1^0 \cdot \mathbf{c}_2^0]_q$, $\hat{\mathbf{c}}_3^1 = [\mathbf{c}_1^0 \cdot \mathbf{c}_2^1 + \mathbf{c}_1^1 \cdot \mathbf{c}_2^0]_q$ and $\hat{\mathbf{c}}_3^2 = [\mathbf{c}_1^1 \cdot \mathbf{c}_2^1]_q$. It follows the multiplication gate with a relinearization operation that takes as input a relinearization key $(\mathbf{rk}_0, \mathbf{rk}_1) = ([-(\mathbf{a} \cdot \mathbf{sk} \pm \mathbf{e}) + \mathbf{sk}^2]_q, \mathbf{a})$ as part of $\mathbf{pk}$. This operation is performed as $\mathbf{c}_3^0 = [\hat{\mathbf{c}}_3^0 + \mathbf{rk}_0 \cdot \hat{\mathbf{c}}_3^2]_q$ and $\mathbf{c}_3^1 = [\hat{\mathbf{c}}_3^1 + \mathbf{rk}_1 \cdot \hat{\mathbf{c}}_3^2]_q$. The final output is a ciphertext $c_r = (\mathbf{c}_3^0, \mathbf{c}_3^1)$.

In both the class of schemes, the decryption stage outputs the correct message only when the error e in the ciphertext being decrypted lies within a predefined error threshold $\pm e_{th}$.

D. Merkle Tree

A Merkle tree [16] (cf. Fig. 1(a)) is a data structure where the leaf nodes store the hash values of the data blocks while the internal nodes store the hash values of their child nodes. The root node, or the Merkle root, stores a single hash value, which forms part of the proof. The data owner generates a Merkle tree by generating hashes of its data elements and then repeatedly generating their pairwise hashes until it is left with only one hash value. The owner stores this hash value on its end and offloads its data along with the generated Merkle tree onto a public system, say a cloud server. Our construction also requires the client to store the hashes of its ciphertexts that are sent to the server. Moreover, unlike the hash function in a

³A bootstrapping key $\mathbf{bk}$ is an encryption of the secret key s under a different secret key s' . Going forward, we assume $\mathbf{bk}$ is a part of $\mathbf{pk}$.

Merkle tree, the hash function in our construction additionally takes an FHE ciphertext as input, and the output hash values link the output of a homomorphic gate to its inputs. The details of our proposed construction are provided in Sect. III.

E. Existing reaction-based attacks

In the literature, *reaction attacks* [12], [13] have been proposed on FHE schemes that utilizes the client as a verification oracle. The authors of [22] proposed a key recovery attack on both the Gentry's scheme [11] and the vDGHV scheme [20]. Their attack works by adding a part of the bootstrapping key⁴ to the ciphertext under evaluation and retrieving the secret bit through the client's reaction. However, this attack does not work for BGV [2] and B/FV [10] schemes as they do not use bootstrapping keys [22]. For FHEW [8] and TFHE [4], the structure of the bootstrapping key and the encrypted message differs from each other. Additionally, the secret key used to generate the bootstrapping key differs from the secret key used to encrypt the message. Therefore, the attack of [22] does not work on these FHE schemes. On the contrary, the authors of [3] proposed a full-key recovery attack that works across all these four FHE schemes that are used in practice. The attack works by adding an error value equal to the error threshold e_{th} to the public component **a** (for LWE-based FHE schemes) or both **a** and **b** (for RLWE-based FHE schemes). When this added error interacts with a secret key bit $sk[i]$ (during decryption), it affects the final noise of the FHE ciphertext only if $sk[i] \neq 0$. Moreover, if the sign of e_{th} coincides with the sign of the error e already present in the FHE ciphertext, it forces the overall error $e_{th} + e$ to cross the threshold e_{th} . This leads to an incorrect decryption, which induces the client to react, thus leaking the value of $sk[i]$.

In a different direction, the authors of [6] provided an idea of possibly recovering the input message of the client. The attack works by guessing the value of the underlying message of the target input ciphertext and replacing it with an encryption of the guessed value. If this guess turns out to be correct, then it will not affect the final result of the homomorphic computation. On the other hand, if the guess turns out to be incorrect, then the final result will turn out to be wrong. The client will not react in the former case, while it will react in the latter case. Finally, the authors of [14] proposed a message recovery attack on a variant of Smart-Vercauteren [19] scheme. However, the scheme was found to be insecure and thus is no longer in use.

III. VR²FHE : VERIFY - THEN - REPAIR OR REACT

As introduced in Sect. I, reaction attacks on FHE schemes exploit the client's feedback when it finds the decrypted result of the requested homomorphic computation to be incorrect. However, in practice, the client does not know the result of the requested computation beforehand. Without extra checks in place, the received result essentially looks random to the client. To alleviate this problem, the client relies on some predefined application-level constraints (ALC) [3], [21]. Such constraints form a sub-circuit of the overall circuit that is to be evaluated

by the server. This sub-circuit takes the decrypted result of the homomorphic computation as input and outputs a single bit denoting whether the constraint is satisfied or not. The client does not provide feedback if the constraint is satisfied; else, it provides one.

For clarity, consider a scenario where the client offloads its encrypted database on a public server and later queries it with encrypted parameters. The server homomorphically evaluates the query and returns the still-encrypted resultant rows to the client, along with a homomorphically computed count of the rows returned. Upon receipt, the client matches the decrypted row-count value with the number of received rows to ensure no rows are missing. This acts as the ALC, since the count matches as long as the query is evaluated correctly. One can observe that the size of this ALC circuit will be much smaller than the entire homomorphic query processing engine, and thus it can be evaluated by the client on its end. Finally, the server can perform a reaction attack by manipulating the encrypted row-count value to induce a mismatch between the decrypted value and the actual number of rows returned, thereby enticing the client to *react*.

Existing techniques to mitigate such attacks try to propose a unified solution to prevent all types of feedback from the client. However, these solutions rely on heavier proving techniques that further slow down the already slow FHE computations. On the other hand, not all feedback from the client is exploitable in practice. For instance, a client may provide feedback even when the decryption failure was accidental (due to the choice of FHE parameters) and not induced by a malicious server [6]. Moreover, these solutions are built to prevent any reactions from the client, i.e., the client simply aborts the protocol once it determines the accompanying proof to be invalid. As it turns out, this approach does not help the client recover from the decryption failure by repairing the result of the homomorphic computation. Thus, the client is either forced to rely on the malicious server for further computations to obtain the correct result or switch to a different server. While the former approach does not alleviate the problem, the latter approach introduces significant costs incurred to migrate the entire setup.

In this section, we explain our proposed solution, named VR²FHE, that allows the client to determine whether a decryption failure is accidental or intentionally induced by the server. Our solution works as a two-step verification process. In the first step, the client ensures whether it received all the data correctly and there were no transmission errors. To do so, the client recomputes the hashes of the received ciphertexts and matches them with the corresponding hashes received from the server. If a mismatch is found, then the client outright rejects the received data and asks the server to retransmit the values. Otherwise, the client proceeds with the second step, where it performs part of the homomorphic computation on its end using the received ciphertext values. As we explain in Sect. IV, this recomputation is limited to only a few layers of the FHE circuits (in some cases, it is just one layer), which is significantly cheaper than running the entire FHE circuit at the client's end. Once done, the locally computed results are matched with the received values to detect any mismatch. A mismatch at this point indicates malicious behaviour by

⁴Each ciphertext in the bootstrapping key encrypts one bit of the secret key.

the server⁵; otherwise, the decryption failure is deemed to be accidental. In the former case, the client performs a `repair` operation by decrypting the resultant ciphertext obtained with the local computation. In the latter case, the client asks the server to perform the computations again and transmit the obtained result. While such re-computation or re-transmission requests are deemed to be *reactions* from the client, we investigate to ensure that they are not exploitable in practice.

We now consider a scenario where the client asks the server to homomorphically evaluate a function f over a vector of input ciphertexts $\mathbf{c}$. Without loss of generality, we assume that f is represented as a layered circuit, as described in Sect. III-A. Using this circuit description, we describe our method in Sect. III-B and establish its security in Sect. III-C.

A. Layered circuits for homomorphic computations

We assume the function f to be evaluated homomorphically can be represented as a circuit comprising of gates. These gates take as input either LWE ciphertexts of the form $c = (\mathbf{a}, b)$ or RLWE ciphertexts of the form $c = (\mathbf{b}, \mathbf{a})$. These gates perform either logical (XOR, AND, etc.) or arithmetic (addition or multiplication) operations over the input ciphertexts depending upon whether we are working over LWE- or RLWE-based FHE schemes. Irrespective of the setting, these gates are followed by ciphertext-maintenance operations such as key-switching or bootstrapping. However, we assume these operations to be part of the gate computation itself. Finally, we assume that the circuit is logically divided into L layers, which takes the form

$$C_L(C_{L-1}(\dots(C_2(C_1(\cdot))))))$$

where each subcircuit C_i takes `inputs(i)` number of ciphertexts as inputs and generates `outputs(i+1)` number of ciphertexts as outputs, with $1 \leq i \leq L$, which form the input to the next subcircuit C_{i+1} .

B. Our vr^2FHE solution

Given an FHE scheme $\text{FHE} = (\text{FHE.Gen}, \text{FHE.Enc}, \text{FHE.Eval}, \text{FHE.Dec})$ and a cryptographically secure hash function $\mathcal{H} : \{0, 1\}^* \rightarrow \{0, 1\}^\ell$, our vr^2FHE method is denoted as $\text{FHE}' = (\text{FHE}'.\text{Setup}, \text{FHE}'.\text{Enc}, \text{FHE}'.\text{Eval}, \text{FHE}'.\text{Verify})$, where the algorithms are defined as follows:

1. $\text{FHE}'.\text{Setup}(1^\lambda, f)$: On input the security parameter λ and a function f that takes `inputs(1)` number of ciphertexts as inputs, our setup algorithm constructs a layered circuit of the form $C_L(C_{L-1}(\dots(C_2(C_1(\cdot))))))$ that evaluates $f(\cdot)$. It also executes $\text{FHE.Gen}(1^\lambda)$ to obtain a pair of keys (pk, sk) . It publishes $(pk, \{C_1, \dots, C_L\})$ and keeps sk as its secret key.
2. $\text{FHE}'.\text{Enc}(pk, x)$: On input public key pk and a message x , our encryption algorithm executes $\text{FHE.Enc}(pk, x)$ to obtain a ciphertext c_x . It further

executes $\mathcal{H}(c_x)$ to obtain its hash h_x . It outputs the ciphertext-hash pair (c_x, h_x) .

3. $\text{FHE}'.\text{Eval}(pk, \mathbf{c}_0, \mathbf{h}_0)$: For a layered circuit $C_L(C_{L-1}(\dots(C_2(C_1(\cdot))))))$, upon input a vector of ciphertexts $\mathbf{c}_1$ and their corresponding hashes $\mathbf{h}_1$, with the dimension of both vectors being `inputs(1)`, our evaluation algorithm works as follows:
For each layer $i = 1$ to $i = L$:

- 3.1. Evaluate $\mathbf{c}_{i+1} \leftarrow \text{FHE.Eval}(pk, C_i, \mathbf{c}_i)$, where $\mathbf{c}_i$ and $\mathbf{c}_{i+1}$ are the input and output ciphertexts, resp., of the subcircuit C_i .
- 3.2. Evaluate $\mathbf{h}_{i+1} \leftarrow \mathcal{H}(\mathbf{c}_{i+1} || \mathbf{h}_i)$, where $\mathbf{h}_i$ denotes the hash of the input ciphertexts $\mathbf{c}_i$ of the subcircuit C_i . For subcircuit C_1 , the hash $\mathbf{h}_1$ is provided by the client. For layers 2 to L , the hashes are generated during the computation itself.

Once all the subcircuits are evaluated, return all ciphertexts $\{\mathbf{c}_i\}$ along with their hashes $\{\mathbf{h}_i\}$.

4. $\text{FHE}'.\text{Verify}(pk, sk, n, \{\mathbf{c}_i\}, \{\mathbf{h}_i\})$: For the same layered circuit $C_L(C_{L-1}(\dots(C_2(C_1(\cdot))))))$ from our evaluation algorithm, upon input the public-secret key pair (pk, sk) , a value n ($1 \leq n \leq L$) that determines the number of circuit levels that are to be recomputed at the client's end⁶, and all the ciphertexts and hashes $(\{\mathbf{c}_i\}, \{\mathbf{h}_i\})$ generated during the evaluation, our verification algorithm works as follows:

Step 1: For each layer $i = n$ to $i = L$:

- 4.1. Evaluate $\mathbf{h}'_{i+1} \leftarrow \mathcal{H}(\mathbf{c}_{i+1} || \mathbf{h}_i)$ and compare with the received $\mathbf{h}_{i+1}$. Output `accept` if all the hashes match; otherwise, output `reject`.

Step 2: For each layer $j = n$ to $j = L$:

- 4.1. Evaluate $\mathbf{c}_{j+1} \leftarrow \text{FHE.Eval}(pk, C_j, \mathbf{c}_j)$, where $\mathbf{c}_j$ and $\mathbf{c}_{j+1}$ are the input and output ciphertexts, resp., of the subcircuit C_j .
- 4.2. Evaluate $\mathbf{h}''_{j+1} \leftarrow \mathcal{H}(\mathbf{c}_{j+1} || \mathbf{h}_j)$, where $\mathbf{h}_j$ denotes the hash of the input ciphertexts $\mathbf{c}_j$ of the subcircuit C_j .
- 4.3. Compare $\mathbf{h}''_{j+1}$ with the received $\mathbf{h}_{j+1}$. Output `accept` if all the hashes match; else, output `reject`.

If the first step outputs `accept` or the second step outputs `reject`, evaluate $x \leftarrow \text{FHE.Dec}(sk, \mathbf{c}_L)$ to obtain the final (decrypted) result. Here $\mathbf{c}_L$ is the final output of the FHE circuit evaluated at the client side.

One may observe that a trivial way to differentiate between an accidental and a malicious decryption failure is by setting n to be 1, i.e., evaluating the entire circuit at the client's end and comparing the resultant ciphertexts with the ones received from the server. Unfortunately, this defeats the entire purpose of outsourced computation, as the client can obtain the results independently of the server. However, in practice, the client needs to verify only a tiny portion of C on its end that comprises entirely of the ALC (more details in Sect. IV). This, in turn, significantly reduces the amount of client-side computation. For other types of failures, such as accidental or transmission, the client simply asks for a recomputation, as

⁵As the evaluation algorithm is deterministic, two runs of the same computation give the same result; thus, a mismatch indicates different evaluations.

⁶These levels are required to be recomputed to differentiate between an accidental and a malicious decryption failure.

these failures do not depend upon the secret key.

C. Security Analysis of vr^2FHE

We now provide a formal security analysis of our proposed method. We begin with a definition of our FHE client FHE_{client} which is exploited as a verification oracle under the IND-CVA1 security notion [6] to carry out interactive key recovery attacks. The reason we focus on the IND-CVA1 notion and not the stronger IND-CCA1 notion is that it is well-known in literature that the existing practical FHE schemes can be broken if given access to an unrestricted decryption oracle under the IND-CCA1 notion [6]. Such an oracle can decrypt any ciphertext of the adversary's choice and directly output the decrypted plaintext value. Therefore, a new line of research [3], [6], [14], [22] is focusing on evaluating the security of practical FHE schemes under the IND-CVA1 notion, which grants the adversary access to a verification oracle. Upon input a ciphertext c , the verification oracle outputs a single bit that denotes whether c decrypts to a correct value or not.⁷

We then define our modified FHE client $^{mod}FHE_{\text{client}}$ that is constructed by incorporating our verification method. Finally, we evaluate the security of our proposed method in the presence of a malicious adversary. We prove that if such an adversary exists, then it can be utilized as a black box to break the second pre-image or collision resistance property of the chosen hash function used in the modified FHE client. We now define the original construction of FHE_{client} as follows.

Definition 2. Our FHE client (FHE_{client}) is a tuple of algorithms ($FHE.Gen$, $FHE.Enc$, $FHE.Verify$) such that:

1. $FHE.Gen$ and $FHE.Enc$ follow from Def. 1.
2. $FHE.Verify(c, m)$: On input an FHE ciphertext c and corresponding message m , the deterministic verification algorithm executes $FHE.Dec(sk, c)$ to obtain the message m' . It outputs 0 if $m' = m$, and 1 otherwise.

Note that the aforementioned definition of our FHE client (FHE_{client}) only contains the key generation, encryption, and verification algorithms and not the evaluation algorithm. The reason is that in the traditional FHE setting, the evaluation is done only on the server side and not on the client side. We now define our IND-CVA1 security game $\Psi_{\text{IND-CVA1}, \mathcal{A}}(1^\lambda)$ between a challenger $\mathcal{Ch}$ and a PPT adversary $\mathcal{A}$ as follows.

IND-CVA1 experiment $\Psi_{\text{IND-CVA1}, \mathcal{A}}(1^\lambda)$:

Query Phase

1. The adversary $\mathcal{A}$ queries the FHE_{client} on a series of ciphertexts $(c_i, \dots, c_j)$, $i \leq j$. The FHE_{client} invokes $FHE.Verify$ on these ciphertexts to obtain a series of verification bits $(vb_i, \dots, vb_j)$, which it sends to $\mathcal{A}$. At this point, we assume that FHE_{client} knows the correct decryption(s) of the received ciphertexts $(c_i, \dots, c_j)$.

Challenge Phase

1. The adversary $\mathcal{A}$ outputs two messages $(x_0, x_1) \in \mathcal{M}$ of equal length and sends them to the challenger $\mathcal{Ch}$.

2. The challenger $\mathcal{Ch}$ uniformly samples bit $bt \leftarrow \{0, 1\}$ and provides $\mathcal{A}$ with a ciphertext $c_{bt} = FHE.Enc(pk, x_{bt})$.

Output Phase

1. With no further access to FHE_{client} , $\mathcal{A}$ outputs a bit bt' .
2. The game outputs 1 if $bt' = bt$, and 0 otherwise.

Now, the advantage of $\mathcal{A}$ winning the IND-CVA1 game is

$$Adv_{\text{IND-CVA1}}^{\mathcal{A}} = |\Pr[\Psi_{\text{IND-CVA1}, \mathcal{A}}(1^\lambda) = 1 | FHE_{\text{client}}] - \frac{1}{2}|$$

where the probability is taken over the randomness used by $\mathcal{A}$ and by the game (in $FHE.Gen$ and $FHE.Enc$). An FHE scheme is said to be IND-CVA1 secure if

$$Adv_{\text{IND-CVA1}}^{\mathcal{A}} \leq \text{negl}(\lambda)$$

However, as demonstrated in a series of works [3], [6], [14], [22], $\mathcal{A}$ can utilize the verification bits $(vb_i, \dots, vb_j)$ to construct an efficient key recovery algorithm $\mathcal{Alg}$ to recover the underlying FHE secret key sk with high probability. Once obtained, $\mathcal{A}$ can use this sk to execute $FHE.Dec(sk, c_{bt})$, obtain the message x_{bt} , and compare it with x_0 and x_1 to reveal bt successfully. This gives $\mathcal{A}$ a non-negligible advantage of winning the IND-CVA1 game.

Modified FHE Client: To prevent the key recovery attacks by utilizing the FHE_{client} , we modify our FHE client by augmenting $FHE.Verify$ into $FHE.Verify'$ and introduce another algorithm $FHE.Hash$ as follows:

Definition 3. Our modified FHE client, $^{mod}FHE_{\text{client}}$, is a tuple of algorithms ($FHE.Gen$, $FHE.Enc$, $FHE.Eval$, $FHE.Hash$, $FHE.Verify'$) such that:

1. $FHE.Gen$, $FHE.Enc$ and $FHE.Eval$ follow as per Def. 1. The $^{mod}FHE_{\text{client}}$ uses $FHE.Eval$ to perform local gate evaluations for verification of results.
2. $FHE.Hash(s)$: On input a string s of arbitrary length, the deterministic hashing algorithm outputs a string h_s of fixed length ℓ , as defined in Sect. II-A2. Note that this function is not evaluated homomorphically, and both the input to and output of this function are plaintext strings.
3. $FHE.Verify'(f, pk, c_{in}, c_{out}, h_{in}, h_{out}, m_{out})$: On input a homomorphic gate f , public key pk , ciphertexts c_{in} and c_{out} along with their corresponding hashes h_{in} and h_{out} , and the message m_{out} , the modified verification algorithm works as follows:
 - 3.1. It computes $h'_{in} = FHE.Hash(c_{in})$ and $h'_{out} = FHE.Hash(c_{out} || h_{in})$.
 - 3.2. If $h'_{in} \neq h_{in}$ or $h'_{out} \neq h_{out}$, it outputs $\perp$, otherwise, go to step 3.3.
 - 3.3. It invokes $FHE.Verify(c_{out}, m_{out})$, where m_{out} is the corresponding plaintext of c_{out} , and obtains as output either 0 or 1. Refer to Def. 2 for details of $FHE.Verify$.
 - 3.4. If $FHE.verify$ outputs 0, then $FHE.Verify'$ outputs 0, otherwise, go to step 3.5.
 - 3.5. It computes $c'_{out} = FHE.Eval(pk, f, c_{in})$.
 - 3.6. It computes $h''_{out} = FHE.Hash(c'_{out} || h_{in})$.
 - 3.7. If $h'_{out} = h''_{out}$, it outputs 1, and $\perp$ otherwise.

Note that contrary to our original FHE client (FHE_{client}), our modified FHE client $^{mod}FHE_{\text{client}}$ also includes the FHE

⁷We refer the readers to [3] and [6] for a more detailed discussion on this.

evaluation algorithm. The reason is that our construction involves running a few FHE computations on the client side during the verification process to aid in the detection of a malicious perturbation. One may observe that our construction assumes the server perturbs only the input and output ciphertexts, c_{in} and c_{out} , of a homomorphic gate, and not any intermediate ciphertexts generated during gate computation. This is because perturbations to intermediate ciphertexts may or may not influence c_{out} . If c_{out} remains unaffected, then decryption succeeds and therefore the client does not react; if it is affected, the result is equivalent to directly perturbing c_{out} . In either case, the server gains no additional advantage, and such scenarios are therefore not considered. Finally, we note that the above definition of our construction also extends to the case where f is a circuit instead of a single gate. The difference is that $\text{FHE.Verify}'$ is now sequentially applied to each gate on a layer-by-layer basis.

IND-CVA1 experiment for modified FHE client. One can observe that the modified FHE client ($\text{modFHE}_{\text{client}}$) now outputs an additional $\perp$ symbol when it detects an intentional decryption failure, and thus it simply aborts the protocol. On the other hand, it outputs a 0 or 1 if it fails to detect an intentional decryption failure. Thus, the goal of the adversary is to trick the $\text{modFHE}_{\text{client}}$ to output a 0 or 1 to achieve the same non-negligible advantage as the original IND-CVA1 game. We now formally define the security of our construction against such adversaries.

Definition 4. Our construction $\text{modFHE}_{\text{client}} = (\text{FHE.Gen}, \text{FHE.Enc}, \text{FHE.Eval}, \text{FHE.Hash}, \text{FHE.Verify}')$ is secure under the IND-CVA1 notion if for all PPT adversaries $\mathcal{A}$, there exists a negligible function negl such that, $\forall \lambda$,

$$|\Pr[\Psi_{\text{IND-CVA1}, \mathcal{A}}(1^\lambda) = 1 \mid \text{modFHE}_{\text{client}}] - \frac{1}{2}| \leq \text{negl}(\lambda)$$

The security of our construction depends on the ability of the verification algorithm $\text{FHE.Verify}'$ to detect any modification in either the input or output ciphertexts of a homomorphic gate and thus output $\perp$. As highlighted in Def. 3, the algorithm relies on the hashes of these ciphertexts to detect such modifications. Thus the adversary $\mathcal{A}$ has to generate the hash first and then perturb the ciphertext. The reason is that the verification algorithm will detect the perturbation in all other cases due to a hash mismatch between the locally computed ciphertext and the one sent by the $\mathcal{A}$. We now present two different scenarios in which $\mathcal{A}$ can first generate the hashes of either the input or output and then perturb either of them.

Evaluate-Hash-Perturb: The adversary first applies the hash function $\mathcal{H}$ on $c_{out} \parallel h_{c_{in}}$ to obtain $h_{c_{out}}$. Here, c_{in} and c_{out} are, resp., the input and output ciphertexts, and $h_{c_{in}}$ is the hash of c_{in} . It now comes up with another ciphertext c'_{out} such that $\mathcal{H}(c'_{out} \parallel h_{c_{in}}) = h_{c_{out}}$. Once done, it sends the values $(c'_{out}, h_{c_{out}}, c_{in}, h_{c_{in}})$ to the $\text{modFHE}_{\text{client}}$.

Hash-Perturb-Evaluate: The adversary first comes up with a different c'_{in} with the same hash value $h_{c_{in}}$. It then computes the homomorphic operation on c'_{in} to obtain c'_{out} and compute the hash $h'_{c_{out}} = \mathcal{H}(c'_{out} \parallel h_{c_{in}})$. Once done, it sends the values $(c'_{out}, h'_{c_{out}}, c'_{in}, h_{c_{in}})$ to the $\text{modFHE}_{\text{client}}$.

Here, we assume that $\mathcal{A}$ has some method that it can use to find another c'_{out} or c'_{in} such that $\mathcal{H}(c'_{out} \parallel h_{c_{in}}) = \mathcal{H}(c_{out} \parallel h_{c_{in}})$ and $h_{c'_{in}} = h_{c_{in}}$, resp. We now propose the following theorem to prove that such a method cannot exist in practice, assuming $\mathcal{H}$ is collision resistant, thereby proving that $\text{modFHE}_{\text{client}}$ is secure under the IND-CVA1 notion.

Theorem 1. Our proposed countermeasure vr^2FHE (Sect. III), represented as $\text{modFHE}_{\text{client}}$, is secure as per Def. 4 under the assumption that $\mathcal{H}$ is a collision resistant hash function.

Proof: Let us assume that $\mathcal{A}$ is successful in breaking the security of vr^2FHE . In other words, it is successfully able to trick the $\text{modFHE}_{\text{client}}$ to output 0 or 1 instead of $\perp$, as shown under Evaluate-Hash-Perturb and Hash-Perturb-Evaluate. We show that there exists a PPT adversary $\mathcal{B}$ that can use $\mathcal{A}$ as a black box to break the second pre-image resistance or collision-resistance property of $\mathcal{H}$ with non-negligible advantages ϵ_1 and ϵ_2 , resp.

1. Hash-Perturb-Evaluate: Let $\mathcal{B}$ be a PPT adversary against the second pre-image resistance property of $\mathcal{H}$.
 - 1.1. Upon receiving message $c_{in} \leftarrow \mathcal{C}$ and its corresponding hash $h_{c_{in}} = \mathcal{H}(c_{in})$, the adversary $\mathcal{B}$ forwards the pair $(c_{in}, h_{c_{in}})$ to the adversary $\mathcal{A}$.
 - 1.2. The adversary $\mathcal{A}$ under Hash-Perturb-Evaluate samples $c'_{in} \leftarrow \mathcal{C}$.
 - 1.3. The adversary $\mathcal{A}$ invokes $\text{FHE.Eval}(pk, f, c'_{in})$ and obtains as output c'_{out} . It additionally computes $h'_{c_{out}} = \mathcal{H}(c'_{out} \parallel h_{c_{in}})$. It sends as output the tuple $(c'_{out}, h'_{c_{out}}, c'_{in}, h_{c_{in}})$ to the $\text{modFHE}_{\text{client}}$.
 - 1.4. The $\text{modFHE}_{\text{client}}$ outputs a bit $bt \neq \perp$.
 - 1.5. On receiving bt , the adversary $\mathcal{A}$ forwards the tuple $(c'_{out}, h'_{c_{out}}, c'_{in}, h_{c_{in}})$ to the adversary $\mathcal{B}$.

The adversary $\mathcal{B}$ succeeds in finding a second pre-image of $(c_{in}, \mathcal{H}(c_{in}))$ as c'_{in} , which is a contradiction. Thus, $\epsilon_1 \leq \text{negl}_1(\lambda)$ since $\mathcal{H}$ is second pre-image resistant.
2. Evaluate-Hash-Perturb: Let $\mathcal{B}$ be a PPT adversary against the collision resistance property of $\mathcal{H}$.
 - 2.1. Upon receiving message $c_{in} \leftarrow \mathcal{C}$ and its corresponding hash $h_{c_{in}} = \mathcal{H}(c_{in})$, the adversary $\mathcal{B}$ forwards the pair $(c_{in}, h_{c_{in}})$ to the adversary $\mathcal{A}$.
 - 2.2. The adversary $\mathcal{A}$ under Evaluate-Hash-Perturb invokes $\text{FHE.Eval}(pk, f, c_{in})$ and obtains as output c_{out} . It additionally computes $h_{c_{out}} = \mathcal{H}(c_{out} \parallel h_{c_{in}})$. Finally, it samples $c'_{out} \leftarrow \mathcal{C}$. It sends as output the tuple $(c'_{out}, h_{c_{out}}, c_{in}, h_{c_{in}})$ to the $\text{modFHE}_{\text{client}}$.
 - 2.3. The $\text{modFHE}_{\text{client}}$ outputs a bit $bt \neq \perp$.
 - 2.4. On receiving bt , the adversary $\mathcal{A}$ forwards the tuple $(c'_{out}, h_{c_{out}}, c_{in}, h_{c_{in}})$ to the adversary $\mathcal{B}$.
 - 2.5. The adversary $\mathcal{B}$ also invokes $\text{FHE.Eval}(ek, f, c_{in})$ and obtains as output c_{out} .

The adversary $\mathcal{B}$ succeeds in finding a collision in the form of c_{out} and c'_{out} since $c_{out} \neq c'_{out}$ but $\mathcal{H}(c_{out} \parallel h_{c_{in}}) = \mathcal{H}(c'_{out} \parallel h_{c_{in}})$, which is a contradiction. Thus, $\epsilon_2 \leq \text{negl}_2(\lambda)$ since $\mathcal{H}$ is collision resistant.

As the advantage of $\mathcal{A}$ winning the IND-CVA1 game against $\text{modFHE}_{\text{client}}$ is the same as the advantage of $\mathcal{B}$ breaking the

second pre-image resistance or collision-resistance property of $\mathcal{H}$, which is $\epsilon_1 + \epsilon_2 \leq \text{negl}(\lambda)$,

$$|\Pr[\Psi_{\text{IND-CVAL}, \mathcal{A}}(1^\lambda) = 1] \mid^{\text{mod FHE}_{\text{client}}} - \frac{1}{2}] \leq \text{negl}(\lambda)$$

This completes our proof. $\blacksquare$

IV. OPTIMIZING VR²FHE

As shown in Sect. III, protecting the entire FHE circuit using our vr²FHE countermeasure ensures its security against reaction-based key recovery attacks, provided a cryptographically secure hash function is used. However, this requires sending all the ciphertexts and their corresponding hashes to the client as part of the proof. In this section, based on a number of observations, we show that it is sufficient to protect a portion of the FHE circuit to thwart practical reaction-based key recovery attacks. This drastically reduces the amount of information that needs to be sent as part of the proof. We now enumerate our observations and show how they can be utilized to optimize our proposed method.

Observation 1: Reaction-based attacks require feedback from the client. Reaction-based attacks exploit the fact that the client sends feedback to the server when it finds the (post-decrypted) result of the requested computation to be incorrect. However, in the practical setting, the client does not know this result beforehand. Therefore, the client is forced to rely on *application-level constraints* (ALC) [3], [21] to differentiate between a correct result and a random value (cf. Sect. III). One can assume this constraint is some function g that forms a small portion of the overall application. Once the client receives the resultant ciphertexts c_r , it evaluates g on the (post-decrypted) result m_r . The client accepts the result if $g(m_r) = 0$ (the constraint is satisfied); else, it provides feedback to the server.

Since the client relies on the output of the ALC to accept or reject the result, the server targets them to obtain feedback from the client. A straightforward way to do so is by tampering with the ciphertexts c_r to generate c'_r . If successful, c'_r decrypts to m'_r and $g(m'_r) = 1$, in which case the client provides feedback. Otherwise, c'_r decrypts to m_r and $g(m_r) = 0$, in which case the client does not provide feedback. Tampering with any other ciphertexts provides no benefit since the client cannot observe the effect of the tampering on its end [3]. Hence, the server is forced to tamper with only those ciphertexts that correspond to the input of the ALC. Based on these observations, we propose protecting only the circuit that corresponds to the constraint function g , which requires the server to send only the proofs corresponding to the inputs of g . Since this function occupies a tiny portion of the overall circuit, it leads to a drastic reduction in the network overhead.

Unfortunately, for the attack demonstrated in [22], we can do no better than protecting the entire circuit corresponding to the ALC. The reason is that, as described in Sect. II-E, this attack works by adding a part of the bootstrapping key $bk[i]$ (that encrypts the secret key bit $sk[i]$) to the input ciphertext and observing its effect on the output. The output does not change if $bk[i]$ encrypts a 0, and it changes if $bk[i]$ encrypts a 1. However, in practice, the adversary is not restricted to

adding $bk[i]$ to only the input ciphertext and can add the same to the output as well as the intermediate ciphertexts. Moreover, the extent of its effect on the final ciphertext further depends on the operation being performed on the modified ciphertext. For brevity, let us assume a single gate operation g_j is being performed on the ciphertexts c_{j1} and c_{j2} , that encrypts messages m_{j1} and m_{j2} . This gate generates the resultant ciphertext as c_{jr} which decrypts to m_{jr} . We assume the adversary adds $bk[i]$ to c_{j1} , and the gate computation $g_j(c_{j1}, c_{j2})$ results in c'_{jr} . If $bk[i]$ encrypts 0, then c'_{jr} decrypts to m_{jr} . On the other hand, if $bk[i]$ encrypts 1, then c'_{jr} decrypts to $m_{jr} + 1$ if g_j denotes an addition operation and to $m_{jr} \cdot m_{j2}$ if g_j denotes a multiplication operation. We can observe that if the underlying message of the perturbed ciphertext changes, then its effect propagates to the output level irrespective of the level at which the perturbation is introduced. Moreover, the probability of the attack's success remains the same across all the levels. Therefore, protection against this attack requires protecting all the levels of the ALC.

Similarly, for the attack on BGV and B/FV schemes demonstrated in [3], we require protecting the entire circuit corresponding to the ALC if it is entirely made up of addition gates. This is due to the inherent design of the addition gate, which ensures that the error propagates to the higher levels without any change in its value. For ease of understanding, let us consider a toy example where ciphertexts $c_1 = (\mathbf{b}_1, \mathbf{a}_1) = (b_1^0 + b_1^1 x, a_1^0 + a_1^1 x)$ and $c_2 = (\mathbf{b}_2, \mathbf{a}_2) = (b_2^0 + b_2^1 x, a_2^0 + a_2^1 x)$ are to be added. The addition is computed as $c_3 = c_1 + c_2 = (\mathbf{b}_1 + \mathbf{b}_2, \mathbf{a}_1 + \mathbf{a}_2) = ((b_1^0 + b_2^0) + (b_1^1 + b_2^1)x, (a_1^0 + a_2^0) + (a_1^1 + a_2^1)x) = (b_3^0 + b_3^1 x, a_3^0 + a_3^1 x) = (\mathbf{b}_3, \mathbf{a}_3)$. At this point, if the adversary adds a perturbation ϵ to a component of input ciphertext, say a_1^0 to change it to $a_1^0 = a_1^0 + \epsilon$, then the computation turns out to be $c'_3 = c'_1 + c_2 = (\mathbf{b}_1 + \mathbf{b}_2, \mathbf{a}'_1 + \mathbf{a}_2) = ((b_1^0 + b_2^0) + (b_1^1 + b_2^1)x, (a_1^0 + \epsilon + a_2^0) + (a_1^1 + a_2^1)x) = ((b_1^0 + b_2^0) + (b_1^1 + b_2^1)x, (a_1^0 + a_2^0) + \epsilon + (a_1^1 + a_2^1)x) = (b_3^0 + b_3^1 x, a_3^0 + \epsilon + a_3^1 x) = (\mathbf{b}_3, \mathbf{a}'_3)$. Based on these equations, we observe that the effect of the added error is the same irrespective of whether it is added to the input (ciphertext c_1) or output (ciphertext c_3) of an addition gate. Therefore, an error added to an addition gate at the bottom of the ALC circuit eventually propagates to the top of the circuit, forcing us to protect the entire circuit using our proposed method.

Now, based on further observations, we show that for other attacks on the practical FHE schemes, it is not even required to protect the entire circuit corresponding to the ALC, and we can further reduce the network overhead by utilizing scheme-specific optimizations. Our observations are based on the fact that for practical RLWE-based FHE schemes, the attack of [3]⁸ becomes intractable when performed at the lower levels of the circuit. On the contrary, for practical LWE-based FHE schemes, this attack becomes infeasible to carry out beyond a certain level, as it requires access to an exponential number of ciphertexts which becomes infeasible to obtain in practice. We now enumerate these observations and show how they can be utilized to perform scheme-specific optimizations.

⁸We perform our optimizations based on the attack of [3], as it is the only known reaction-based attack to work on practical FHE schemes.

Observation 2: The attack of [3] becomes intractable for BGV and B/FV schemes when targeting the multiplication operation. Contrary to the addition gate of the BGV and B/FV schemes, the construction of their multiplication gate ensures that a perturbation introduced in its input ciphertext spreads around in its output ciphertext in an intractable manner. For ease of understanding, we consider our previous toy example, where the multiplication $c_1 \times c_2$ is computed as

$$\begin{aligned}
c_3^0 &= \mathbf{b}_1 \cdot \mathbf{b}_2 = (b_1^0 + b_1^1 x) \cdot (b_2^0 + b_2^1 x) \\
&= b_1^0 b_2^0 + b_1^0 b_2^1 x + b_2^0 b_1^1 x + b_2^1 b_1^1 x^2 \\
&= b_1^0 b_2^0 + (b_1^0 b_2^1 + b_2^0 b_1^1) x + b_2^1 b_1^1 x^2 \\
c_3^1 &= \mathbf{b}_1 \cdot \mathbf{a}_2 + \mathbf{b}_2 \cdot \mathbf{a}_1 \\
&= (b_1^0 + b_1^1 x) \cdot (a_2^0 + a_2^1 x) + (b_2^0 + b_2^1 x) \cdot (a_1^0 + a_1^1 x) \\
&= (b_1^0 a_2^0 + b_2^0 a_1^0) + (b_1^0 a_2^1 + a_2^0 b_1^1 + b_2^0 a_1^1 + a_1^0 b_2^1) x \\
&\quad + (a_2^1 b_1^1 + a_1^1 b_2^1) x^2 \\
c_3^2 &= \mathbf{a}_1 \cdot \mathbf{a}_2 = (a_1^0 + a_1^1 x) \cdot (a_2^0 + a_2^1 x) \\
&= a_1^0 a_2^0 + a_1^0 a_2^1 x + a_2^0 a_1^1 x + a_2^1 a_1^1 x^2 \\
&= a_1^0 a_2^0 + (a_1^0 a_2^1 + a_2^0 a_1^1) x + a_2^1 a_1^1 x^2
\end{aligned}$$

From the above equations, one may observe that each component of the input ciphertext is involved in the computation of two of the components of the output ciphertext. At this point, if the adversary adds a perturbation ϵ to a component of any input ciphertext, it spreads across multiple components of the output ciphertext. For example, say ϵ is added to a_1^0 to change it to $a_1^0 = a_1^0 + \epsilon$. The resulting ciphertext now turns out to be

$$\begin{aligned}
c_3^0 &= b_1^0 b_2^0 + (b_1^0 b_2^1 + b_2^0 b_1^1) x + b_2^1 b_1^1 x^2 \\
c_3^1 &= (b_1^0 a_2^0 + b_2^0 a_1^0 + b_2^0 \epsilon) \\
&\quad + (b_1^0 a_2^1 + a_2^0 b_1^1 + b_2^0 a_1^1 + a_1^0 b_2^1 + b_2^1 \epsilon) x \\
&\quad + (a_2^1 b_1^1 + a_1^1 b_2^1) x^2 \\
c_3^2 &= (a_1^0 a_2^0 + a_2^0 \epsilon) + (a_1^0 a_2^1 + a_2^0 a_1^1 + a_2^1 \epsilon) x + a_2^1 a_1^1 x^2
\end{aligned}$$

Based on these equations, we observe that the amount of perturbations added to the components of the output ciphertext differs for each coefficient, which is based on the coefficient of the input ciphertexts with which ϵ interacts. For example, $a_2^0 \epsilon \neq a_2^1 \epsilon$ since $a_2^0 \neq a_2^1$ and thus different amounts of perturbations are added to each coefficient of c_3^2 . Furthermore, each coefficient in the ciphertexts is a large integer drawn from $\mathbb{Z}_q$. Thus, multiplying even a small ϵ with these coefficients results in a blowup of the added perturbation, which further spreads across multiple components of the final ciphertext. Adding an additional perturbation value to counterbalance the effect of ϵ is futile since it similarly spreads around over the other components.⁹ Lastly, the aforementioned computations only generate the intermediate result, and the final result is obtained by relinearizing this value (cf. Sect. II-C), which further involves ciphertext-ciphertext multiplication.

Based on this observation, we propose protecting the levels starting from the last (starting from the bottom) multiplication

⁹One can add $-a_2^1 \epsilon x$ to c_3^2 to cancel out the term $a_2^1 \epsilon x$ as a_2^1 and ϵ are both known. However, this is equivalent to adding $a_2^0 \epsilon$ to c_3^2 .

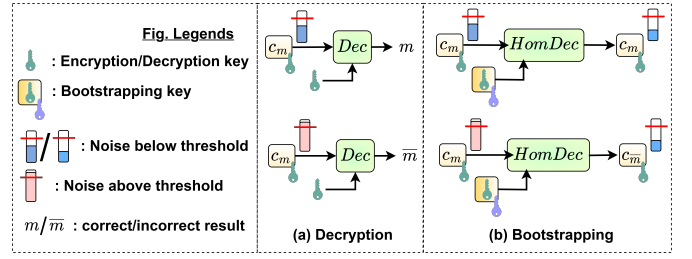


Figure 2: Similarity between (a) decryption and (b) bootstrapping. Bootstrapping outputs an encryption of the wrong message if the total error grows beyond the threshold.

gate in the circuit and all the way up to the topmost level of the circuit. Adding any perturbation below this level will make the attack intractable due to the property of the multiplication gate itself. Finally, given it is common in the FHE literature to end each layer (and in turn the entire circuit) with a multiplication gate [1], it becomes sufficient to protect the topmost layer of the circuit, which requires sending the ciphertexts in the final and penultimate layers, along with the hashes from the top three layers, as part of the proof.

Observation 3: The attack of [3] becomes infeasible for FHEW and TFHE schemes when performed beyond a certain level. In this section, we first show that the attack of [3] on the FHEW scheme cannot be directly performed if the perturbation is done at the lower levels. This is due to the underlying mathematics of the gate computation, which is followed by a bootstrapping step. One might recall from Sect. II-C that bootstrapping involves homomorphically decrypting a ciphertext c using an encryption of $\mathbf{sk}$. Thus, similar to decryption (Fig. 2(a)), bootstrapping outputs an encryption of the incorrect message when the noise in c grows beyond the threshold (Fig. 2(b)). Hence, upon introducing a perturbation at some intermediate ciphertexts, a decryption failure can occur due to the bootstrapping step, which can then propagate to the next levels. We now show why this failure alone cannot be exploited to recover $\mathbf{sk}$ with the help of a toy example.

Let us consider two ciphertexts, $c_1 = (\mathbf{a}_1, b_1 = [\mathbf{a}_1 \cdot \mathbf{sk} + \mu_1 \pm e_1]_q)$ and $c_2 = (\mathbf{a}_2, b_2 = [\mathbf{a}_2 \cdot \mathbf{sk} + \mu_2 \pm e_2]_q)$ where μ_1 and μ_2 can be either 0 or 1. Moreover, we take the gate constant as $c_G = (1024, b_{gc}^F)$ where b_{gc}^F is dependent on the gate G being computed (cf. Sect. II-C). As defined in [9], the values of $+e_{th}$ and $-e_{th}$ are 63 and -64 for all the gates and $q = 512$. We evaluate $c_3 = G(c_1, c_2) = (\mathbf{a}_3, b_3)$ as $\mathbf{a}_3 = [(\mathbf{1024} - \mathbf{a}_1 - \mathbf{a}_2)]_q$ and $b_3 = b_{gc}^F - [(b_1 + b_2)]_q = b_{gc}^F - [(\mathbf{a}_1 \cdot \mathbf{sk} + \mu_1 \pm e_1 + \mathbf{a}_2 \cdot \mathbf{sk} + \mu_2 \pm e_2)]_q$. Now, to flip the underlying bit during the bootstrapping stage of the $G(\cdot, \cdot)$ operation, we add e_{th} to the i^{th} component of $\mathbf{a}_1$, i.e., $a_1[i] = a_1[i] + e_{th}$. The phase ϕ_3 of this new ciphertext c'_3 is computed as

$$\begin{aligned}
\phi_3 &= [b_3 - \mathbf{a}'_3 \cdot \mathbf{sk}]_q \\
&= [(b_{gc}^F - \mu_1 - \mu_2) - (\mathbf{a}_1 + \mathbf{a}_2) \cdot \mathbf{sk} \pm (e_1 + e_2) \\
&\quad - (\mathbf{1024} - \mathbf{a}_1 - \mathbf{a}_2) \cdot \mathbf{sk} + (sk[i] \times e_{th})]_q \\
&= [(b_{gc}^F - \mu_1 - \mu_2) \pm (e_1 + e_2) + (sk[i] \times e_{th})]_q \\
&= [(b_{gc}^F - \mu_3) \pm (e_3) + (sk[i] \times e_{th})]_q
\end{aligned}$$

where the additional term $(sk[i] \times e_{th})$ comes from the dot product between the modified a_1 and sk . In the above equation, $1024 \cdot sk$ generates a vector where each element belongs to $\{-1024, 0, 1024\}$ depending on whether the coefficient of 1024 interacts with -1 , 0 or 1 . Thus, $(1024 \cdot sk) \bmod 512$ generates a vector of all zeroes and therefore is ignored going further. We now see the effect of four different combinations of $sk[i]$ and e_{th} on the decryption during bootstrapping.

① $sk[i] = -1, e_{th} = -e_{th} = -64$:

$$\begin{aligned}\phi_3 &= [(b_{gc}^F - \mu_3) \pm e_3 + (-1 \times -64)]_q \\ &= [(b_{gc}^F - \mu_3 + 64) \pm e_3]_q \\ \frac{\phi_3 + 64}{128} &= \left[\frac{(b_{gc}^F - \mu_3)}{128} + 1.00 \pm \frac{e_3}{128} \right]_q\end{aligned}$$

② $sk[i] = -1, e_{th}^\pm = +e_{th} = 63$:

$$\begin{aligned}\phi_3 &= [(b_{gc}^F - \mu_3) \pm e_3 + (-1 \times 63)]_q \\ &= [(b_{gc}^F - \mu_3 - 63) \pm e_3]_q \\ \frac{\phi_3 + 64}{128} &= \left[\frac{(b_{gc}^F - \mu_3)}{128} + 0.01 \pm \frac{e_3}{128} \right]_q\end{aligned}$$

③ $sk[i] = 1, e_{th}^\pm = -e_{th} = -64$: The situation will be same as case ② and thus is not elaborated.

④ $sk[i] = 1, e_{th}^\pm = +e_{th} = 63$: The situation will be same as case ① and thus is not elaborated.

Thus, the occurrence of decryption failure now depends upon the unknown values of μ_3 and e_3 , and whether e_3 is added or subtracted during the last step of the decryption. Thus the adversary cannot differentiate between when $sk[i] = 1$ and $sk[i] = -1$ if the perturbation is introduced in some intermediate ciphertext and it gets a reaction from the client. This is due to the lack of a unique combination of the value of $sk[i]$ and the sign of e_{th} .

However, this attack can be modified to provide a unique combination of $sk[i]$ and e_{th} which can then be used to differentiate between when $sk[i] = 1$ and $sk[i] = -1$. The new attack works similarly to the attack on RLWE-based FHE schemes from [3]. Concretely, we split the overall perturbation e_{th} into two sub-components ϵ and $e_{th} - \epsilon$. The component ϵ is added to, wlog, $a_1[i]$ while the remaining component $e_{th} - \epsilon$ is either added to or subtracted from b_3 depending on whether we assume $sk[i]$ to be 1 or -1 , and the sign of the error e_3 ¹⁰ to be $+ve$ or $-ve$. We can observe that, similar to [3], when $sk[i] = 1$, the total perturbation being added to b_3 during decryption becomes $e_{th} - \epsilon + (1) \times \epsilon = +e_{th}$. Similarly, when $sk[i] = -1$, the total perturbation being added becomes $-(e_{th} - \epsilon) + (-1) \times \epsilon = -e_{th} + \epsilon - \epsilon = -e_{th}$. Now, apart from $sk[i]$, if the guess about the sign of e_3 also turns out to be correct, then the total error in b_3 during bootstrapping becomes $+e_3 + (+e_{th}) > +e_{th}$ or $-e_3 + (-e_{th}) < -e_{th}$. We can observe that both cases lead to a decryption failure during bootstrapping, since it outputs an encryption of the incorrect message. On the other hand, in case our guess

¹⁰We target the error in the resultant ciphertext c_3 , and not c_1 , since it will be homomorphically decrypted during the bootstrapping operation.

Truth Table of AND			Truth Table of OR			Truth Table of XOR		
m_1	m_2	r	m_1	m_2	r	m_1	m_2	r
1 $\times$	0	0 $\checkmark$	1 $\times$	0	1 $\times$	1 $\times$	0	1 $\times$
1 $\times$	1	1 $\times$	1 $\times$	1	1 $\checkmark$	1 $\times$	1	0 $\times$
0 $\times$	0	0 $\checkmark$	0 $\times$	0	0 $\times$	0 $\times$	0	0 $\times$
0 $\times$	1	0 $\times$	0 $\times$	1	1 $\checkmark$	0 $\times$	1	1 $\times$

(a) AND

(b) OR

(c) XOR

Figure 3: Truth tables of (a) AND, (b) OR, and (c) XOR gates. m_1 and m_2 are the incorrect and correct results of the previous gate. $\times$ (red) = incorrect result, $\checkmark$ (green) = correct result.

turns out to be incorrect, i.e., $sk[i]$ is actually -1 when we guessed it to be 1 and vice versa, then the total perturbation becomes $e_{th} - \epsilon + (-1) \times \epsilon = +e_{th} - 2\epsilon < +e_{th}$ and $-(e_{th} - \epsilon) + (1) \times \epsilon = -e_{th} + \epsilon + \epsilon = -e_{th} + 2\epsilon > -e_{th}$. In both cases, one can observe that the overall error in b_3 , even after accounting for the value of e_3 , does not cross the overall error threshold and therefore does not lead to a decryption failure. This provides a unique combination of $sk[i]$ and e_{th} , which can now be used to carry out the attack at the lower levels.

Now, contrary to FHEW, TFHE uses a binary key, i.e., from $\{0, 1\}$, and therefore does not require further differentiation between 1 and -1 . In other words, feedback from the client, irrespective of whether e_{th} that is added to the ciphertext is positive or negative, denotes that the targeted secret bit is 1. Therefore, in the case of TFHE, the attack of [3] can directly be carried out at the lower levels without any modifications.

Nonetheless, irrespective of the underlying FHE scheme, the client only decrypts the final resultant ciphertexts of a homomorphic computation and not the intermediate ciphertexts. Therefore, a successful perturbation at some lower level further needs to propagate to the output ciphertexts in order for it to be exploitable. However, this reduces the success probability of an attacker as it goes down deeper into the circuit due to the interference from the other intermediate ciphertexts, which in turn increases the number of ciphertexts that are required to carry out the attack. Now, the successful propagation of the perturbation to the output of a gate computation $G(c'_1, c_2)$, where c'_1 is the perturbed ciphertext, depends on both the gate G and the bit value m_2 underlying c_2 . In the case of the XOR gate, the perturbation propagates irrespective of the value of m_2 (Fig. 3(c)), while for the AND (Fig. 3(a)) and OR (Fig. 3(b)) gates, it only propagates when m_2 is 1 and 0, resp. We now determine the success probability of an adversary as it goes down from level L (output) to level 1 (input), considering a circuit made entirely of AND, OR, and NOT gates. We focus on the top 4 levels (cf. Fig. 4), and provide a recurrence relation to compute the success probabilities for the remaining levels.

Let $\Pr[\mathcal{P}_\ell]$ denote the probability of decryption failure at level $\ell + 1$ when the perturbation is introduced at level ℓ and $\Pr[\mathcal{M}_{\ell,G} = m]$ denote the probability of an input to a gate G at level ℓ being m (0/1). For each case, $\Pr[\mathcal{P}_\ell] = \frac{1}{4}$ for TFHE and $\Pr[\mathcal{P}_\ell] = \frac{1}{6}$. The reason is that in the case of both TFHE and FHEW, the success of decryption failure depends only on the success of the server in correctly guessing the value of the secret bit and the sign of the error contained in the

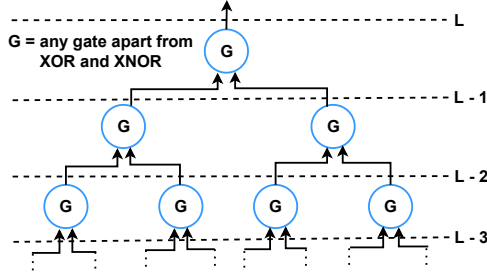


Figure 4: Success probability of attacker when perturbing at different levels (defined based on the ciphertexts and not gates).

targeted ciphertext but not the message underlying the targeted ciphertext.¹¹ Given the secret bit can only take a value from $\{0, 1\}$ for TFHE and $\{0, \pm 1\}$ for FHEW, the server can correctly guess the secret bit with probability $\frac{1}{2}$ and $\frac{1}{3}$ for TFHE and FHEW, resp. Similarly, it can correctly guess the sign of the error with probability $\frac{1}{2}$ since the sign can either be positive or negative. This holds even when the perturbation is introduced at some intermediate ciphertext since decryption failure now occurs due to bootstrapping (Fig. 2(b)). Thus we only compute $\Pr[\mathcal{M}_{\ell,G} = m]$ for different ℓ .

① Level L and L - 1: For both of these cases, $\Pr[\mathcal{P}_L] = \Pr[\mathcal{P}_{L-1}] = \frac{1}{4} = 2^{-2}$ for TFHE and $\Pr[\mathcal{P}_L] = \Pr[\mathcal{P}_{L-1}] = \frac{1}{6} \approx 2^{-3}$ since the perturbed ciphertext will be the final output ciphertext itself. For ciphertexts at level L, decryption failure occurs when the client decrypts the ciphertext at its end, while for level L - 1, it occurs during bootstrapping itself.

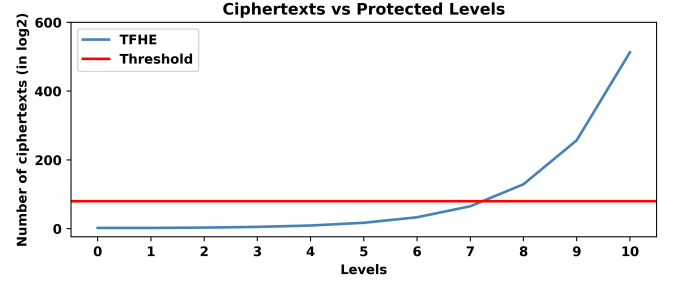
② Level L - 2: For $\ell = L - 2$, $\Pr[\mathcal{P}_{L-2}] = \frac{1}{4}$ for TFHE and $\Pr[\mathcal{P}_{L-2}] = \frac{1}{6}$ for FHEW. Whether this perturbation propagates to level L depends on m_2 which, assuming an average case, needs to be 1 or 0 depending on G being AND or OR, resp. Thus $\Pr[\mathcal{M}_{L-1,G} = m_2] = \frac{1}{2}$, and the success probability turns out to be $\Pr[\mathcal{P}_L] = \Pr[\mathcal{P}_{L-2}] \times \Pr[\mathcal{M}_{L-1,G} = m_2]$. For TFHE, this value turns out to be $\frac{1}{4} \times \frac{1}{2} = \frac{1}{8} = 2^{-3}$, while for FHEW, this value turns out to be $\frac{1}{6} \times \frac{1}{2} = \frac{1}{12} \approx 2^{-4}$.

③ Level L - 3: For $\ell = L - 3$, $\Pr[\mathcal{P}_{L-3}] = \frac{1}{4}$ for TFHE and $\Pr[\mathcal{P}_{L-3}] = \frac{1}{6}$ for FHEW. Whether this perturbation propagates to level L - 1 depends on m_2 which, assuming an average case, needs to be 1 or 0 depending on G being AND or OR, resp. Thus $\Pr[\mathcal{M}_{L-2,G} = m_2] = \frac{1}{2}$. Furthermore, whether this perturbation propagates to level L depends on m'_2 which further depends on the inputs to the gate G' that generated it. Now, assuming that for the output of G' to be m'_2 , both its inputs need to be m'_2 , i.e., $\Pr[\mathcal{M}_{L-1,G} = m'_2] = \frac{1}{2} \times \frac{1}{2}$.¹² Thus $\Pr[\mathcal{P}_L] = \Pr[\mathcal{P}_{L-3}] \times \Pr[\mathcal{M}_{L-2,G} = m_2] \times \Pr[\mathcal{M}_{L-1,G} = m'_2]$. For TFHE, this value turns out to be $\frac{1}{4} \times \frac{1}{2} \times \frac{1}{2} \times \frac{1}{2} = \frac{1}{32} = 2^{-5}$, while for FHEW, this value turns out to be $\frac{1}{6} \times \frac{1}{2} \times \frac{1}{2} \times \frac{1}{2} = \frac{1}{48} \approx 2^{-6}$.

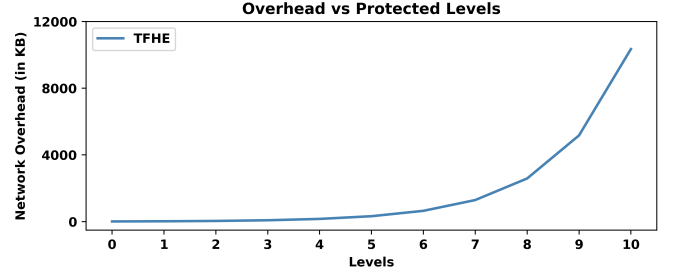
For lower levels, the success probability is computed using the recurrence relations from Eq. 1 and Eq. 2, which correspond to TFHE and FHEW, resp. For both the equations, n is

¹¹The goal of these attacks is to simply change the underlying message [3].

¹²Our assumption is valid since for the output of an AND and OR gate to be 1 and 0, both its inputs needs to be 1 and 0, resp.



(a) Ciphertexts to recover 1-bit of secret key



(b) Network overhead

Figure 5: Plot of (a) number of ciphertexts required to recover 1-bit of secret key vs level at which perturbation is introduced, and (b) network overhead in terms of data transmitted to client vs level till which protection is ensured. The horizontal line in (a) represents the number of ciphertexts reaching 2^{80} , making it infeasible for the adversary to carry out the attack in practice.

the level where perturbation is introduced with $n = 1$ being level L. The final probability turns out to be $2^{-F(n)}$.

$$F(n) = \begin{cases} 2F(n-1) - 1, & \text{if } n > 2. \\ 2, & \text{if } n = 1 \text{ or } n = 2. \end{cases} \quad (1)$$

$$F(n) = \begin{cases} 2F(n-1) - 2, & \text{if } n > 2. \\ 3, & \text{if } n = 1 \text{ or } n = 2. \end{cases} \quad (2)$$

Fig. 5 shows the plot of the number of ciphertexts n_c required to successfully recover 1 bit of the secret key when the perturbations are introduced at levels $L - i$, where $0 \leq i \leq 10$. Referring to Fig. 5(a), n_c grows slowly from levels L to L - 7, after which it grows exponentially. Moreover, at level L - 7, n_c increases to 2^{80} (depicted as a red line in Fig. 5(a)) for both FHEW and TFHE. Considering the key sizes of 500 and 630 for FHEW and TFHE, resp., the total number of ciphertexts required to carry out the attack therefore becomes 2^{89} and 2^{90} for FHEW and TFHE, resp., if the perturbation is introduced at level L - 7, which are infeasible to obtain in practice.

Thus, for both schemes, it is sufficient to protect the top 7 levels of the circuit. i.e., from level L to level L - 6. However, the network overhead also increases with the number of protected levels, as shown in Fig. 5(b). The reason is that the amount of auxiliary information to be sent to the client increases in direct proportion to the number of protected levels. Hence, there is a trade-off between the security requirement

Table I: Shows the time required to run the homomorphic gates without and with hash generations. ‘-’: gate is not available.

		NAND	AND	NOR	OR	XNOR	XOR	ADD	MULT
FHEW	Original Gate Computation	0.0787s	0.0792s	0.0792s	0.0782s	-	-	-	-
	Gate Computation with Hashing	0.0791s	0.0793s	0.0794s	0.0791s	-	-	-	-
TFHE	Original Gate Computation	0.0333s	0.0337s	0.0332s	0.0338s	0.0339s	0.0333s	-	-
	Gate Computation with Hashing	0.0335s	0.0338s	0.0334s	0.0339s	0.0344s	0.0334s	-	-
BGV	Original Gate Computation	-	-	-	-	-	-	0.0001s	0.0038s
	Gate Computation with Hashing	-	-	-	-	-	-	0.0077s	0.0049s

and the network overhead. We would like to mention that the aforementioned computations are for the base circuit from Fig. 4. However, our analysis holds for any type of circuit, as they can be thought of as a combination of our base circuit.

◇ **Caveat.** The above analysis works only if the circuit does not contain any XOR and XNOR gates, as they always propagate the perturbation, unlike AND and OR gates. Thus, to mitigate this issue, we propose building the circuit in a sum-of-products (SOP) form that uses only AND, OR, and NOT gates. Moreover, the number of layers to be protected depends on both the bit security that is required and the amount of network overhead that is tolerable. Thus, the clients need to make their own policies in terms of the number of layers to be protected, based on the required amount of security and available network bandwidth. The policy may also enforce a change of the keys once a sufficient number of computation requests are made. For example, a change of keys can be done after $2^{30} \approx 1$ billion computation requests, which makes it sufficient to protect the top six levels.

Proposed framework. Our framework is based on the aforementioned observations that for practical FHE schemes, it is sufficient to protect a small portion of the overall circuit to secure them against reaction-based key recovery attacks. This leads to a significant reduction in the amount of auxiliary information that is sent back to the client as part of proof. Our framework works by taking an FHE scheme and existing reaction-based key recovery attacks as inputs. Then, based on our observations corresponding to the input FHE scheme, it evaluates the number of layers to be protected for each attack vector. Finally, it outputs the maximum value among these layer counts. The reason we go for the maximum value is that in the real-world scenario, the client might not be aware of the attack vector being employed by the malicious server to perform the key recovery attack. Therefore, to ensure protection against the existing most powerful attack vector (which supports the attack till the highest depth), we protect till the level where the attack remains feasible. This further ensures that any other existing less powerful attack vector will be thwarted simultaneously. Finally, one can perform similar analysis for any newer attack vectors that come up in the literature and can update our framework based on the observations made against these vectors.

V. COMPUTATIONAL OVERHEAD

In this section, we calculate the computation overhead of our proposed method on practical FHE schemes. All the experiments were performed on a system running on Intel® Xeon®

Silver 4210R @ 2.4GHz powered by Ubuntu 20.04 LTS. As our hash function $\mathcal{H}$, we used a C-based implementation of SHA-256 from [18].¹³ Moreover, since the ciphertexts are of the form (a, b) for FHEW and TFHE and (b, a) for BGV and B/FV, where each of the entries of the ciphertext is an integer, we first converted these entries into strings using the standard C-function `snprintf()` and then concatenated them using our own function `concat_cipher()` to generate a single string c_s . The hash of c_s was then generated as $h_c = \mathcal{H}(c_s)$. As our hash function takes the entire ciphertext as a concatenated string c_s of its elements, any changes made to the ciphertext alter c_s , which in turn changes the hash of the ciphertext. Moving ahead, to generate the hash h_{c_I} of some intermediate ciphertext c_I using the hashes $h_{c_{I_1}}$ and $h_{c_{I_2}}$, we first concatenated the components of this ciphertext to c_{I_s} using the aforementioned method to generate c'_{I_s} . Here c_{I_1} and c_{I_2} are the ciphertexts upon which a gate computation was performed to obtain c_I . The hash of this ciphertext was then generated as $h_{c_I} = \mathcal{H}(\text{concat}(c'_{I_s}, h_{c_{I_1}}, h_{c_{I_2}}))$, where `concat()` is a custom function that takes three strings as input and concatenates them to produce a single string. Finally, the hash comparison was done in the form of a string comparison using the standard C-function `strcmp()` as the hashes obtained are in the form of strings. Our code is publicly available at <https://github.com/bhuvneshchaturvedi2512/vr2FHE>.

Table I provides a comparison of the total running times of the original gates and when the gate computation is followed by the hash generation step. From the table, we can observe that for FHEW and TFHE, original gate computation takes an average of 0.0788 and 0.0335 seconds, resp., while the average time increases to 0.0792 and 0.0337 seconds, resp., with the inclusion of the hash generation step. Thus the hash generation step increases the per-gate computation time by an average of 0.0004 and 0.0002 seconds, or by 0.5% and 0.6%, for FHEW and TFHE, resp. In case of BGV, we can observe that implementing the multiplication gate with the hash generation step does not lead to a significant increase in the computation time as compared to the original implementation of the gate. On the other hand, we can observe that implementing the addition gate with the hash generation step increases the computation time significantly as compared to the original implementation of the gate. This is mainly due to the implementation level difference between the way the gate is computed and the way the hash is generated, and can be alleviated with an efficient implementation of the hash generation step.

¹³Our proposed method can be instantiated using any cryptographically secure hash function.

VI. DISCUSSION

Message recovery attacks. There have also been works in the literature that propose reaction-based message recovery attacks, which aim to directly recover the inputs to or outputs of a homomorphic computation. In one such work, the authors of [6] proposed a message recovery attack that works by replacing one of the input ciphertexts encrypting an unknown message with the encryption of a known message. If the messages turn out to be the same, it does not alter the result of the computation and therefore does not induce a reaction from the client; otherwise, it leads to an incorrect result, which induces a reaction. Assuming the possible number of message values to be t , the probability that the guess of the message turns out to be correct is $\frac{1}{t}$. Therefore, in the worst case, the server has to test all possible values of t to recover the correct message. The server can then repeat the attack for each input to recover them individually. Assuming m number of input values, the server has to repeat the attack for $m \times t$ times to recover all inputs. Finally, the attack has to be repeated from the start for each new set of inputs, unlike the key recovery attacks, where the key has to be recovered only once which can then be used to recover all the messages.

Other approaches for FHE integrity. Apart from cryptographic techniques, there have also been recent attempts to use tamper-resistant hardware environments such as Trusted Execution Environments (TEEs) to provide integrity to FHE computations [7], [21]. However, these TEEs are interfaced with an untrusted environment which does not offer the same integrity protection as the enclaves. This allows a malicious server to tamper with the data entering into or leaving the TEEs. Furthermore, solutions relying on TEEs need to make an additional trust assumption on the hardware vendor of these enclaves [7], [21], which is often not desirable in practice.

VII. CONCLUSION

While proven to be secure against honest but curious adversaries, the security of FHE schemes falters in the presence of malicious adversaries. As is evident from a line of works named reaction attacks, a malicious server can utilize the FHE client as a verification oracle to perform full-key recovery attacks in practice. While there have been attempts to use cryptographic techniques or tamper-resistant hardware to mitigate these attacks, these solutions either suffer from significant network and computation overhead or require additional trust assumptions on the hardware vendors. In this paper, we propose vr^2FHE , an efficient solution that uses the concept of the Merkle tree. Incorporating this technique leads to a more computationally efficient solution without relying on any additional trust assumptions. We further perform optimizations to make our proposed solution suitable for practical deployment. As the world gears towards a privacy-aware society, real-world applications need to consider newer threat models and adversarial powers for holistic security and privacy guarantees. FHE, being at the center of such a privacy-aware world, requires not only confidentiality but also integrity guarantees for secure assimilation into sensitive applications.

REFERENCES

- [1] S. Atapoor, K. Baghery, H. V. Pereira, and J. Spiessens, “Verifiable fhe via lattice-based snarks,” *IACR Communications in Cryptology*, 2024.
- [2] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, “(leveled) fully homomorphic encryption without bootstrapping,” in *ACM Transactions on Computation Theory (TOCT)*, 2014, pp. 1–36.
- [3] B. Chaturvedi, A. Chakraborty, A. Chatterjee, and D. Mukhopadhyay, ““ask and thou shall receive”: Reaction-based full key recovery attacks on fhe,” in *European Symposium on Research in Computer Security*. Springer, 2024, pp. 457–477.
- [4] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, “Tfhe: fast fully homomorphic encryption over the torus,” *Journal of Cryptology*, pp. 34–91, 2020.
- [5] —, “TFHE: Fast fully homomorphic encryption library,” August 2016, <https://tfhe.github.io/tfhe/>.
- [6] I. Chillotti, N. Gama, and L. Goubin, “Attacking fhe-based applications by software fault injections,” *Cryptology ePrint Archive*, 2016.
- [7] J. Drean, F. Jepsen, E. Suh, S. Devadas, A. Jaleel, and G. Saileshwar, “Teaching an old dog new tricks: Verifiable fhe using commodity hardware,” arXiv preprint arXiv:2412.03550, 2024.
- [8] L. Ducas and D. Micciancio, “FHEW: bootstrapping homomorphic encryption in less than a second,” in *Annual international conference on the theory and applications of cryptographic techniques*. Springer, 2015, pp. 617–640.
- [9] —, “FHEW: A fully homomorphic encryption library,” May 2017, <https://github.com/lducas/FHEW>.
- [10] J. Fan and F. Vercauteren, “Somewhat practical fully homomorphic encryption,” *Cryptology ePrint Archive*, 2012.
- [11] C. Gentry, “Fully homomorphic encryption using ideal lattices,” in *Proceedings of the forty-first annual ACM symposium on Theory of computing*, 2009, pp. 169–178.
- [12] C. Hall, I. Goldberg, and B. Schneier, “Reaction attacks against several public-key cryptosystems,” in *Information and Communication Security: Second International Conference, ICICS’99*. Springer, 1999, pp. 2–12.
- [13] Z. Hu, F. Sun, and J. Jiang, “Ciphertext verification security of symmetric encryption schemes,” *Sci. China Ser. F Inf. Sci.*, pp. 1617–1631, 2009.
- [14] J. Loftus, A. May, N. P. Smart, and F. Vercauteren, “On cca-secure somewhat homomorphic encryption,” in *In Selected Areas in Cryptography*, 2011, pp. 55–72.
- [15] V. Lyubashevsky, C. Peikert, and O. Regev, “On ideal lattices and learning with errors over rings,” in *Advances in Cryptology—EUROCRYPT 2010*. Springer, 2010, pp. 1–23.
- [16] R. C. Merkle, “A digital signature based on a conventional encryption function,” in *Advances in Cryptology — CRYPTO ’87*. Springer, 1987, pp. 369–378.
- [17] O. Regev, “On lattices, learning with errors, random linear codes, and cryptography,” *Journal of the ACM (JACM)*, pp. 1–40, 2009.
- [18] P. repository, “Public domain c sha-256 implementation,” <https://github.com/983/SHA-256>, accessed: 2024-11-10.
- [19] N. P. Smart and F. Vercauteren, “Fully homomorphic encryption with relatively small key and ciphertext sizes,” in *International Workshop on Public Key Cryptography*. Springer, 2010, pp. 420–443.
- [20] M. Van Dijk, C. Gentry, S. Halevi, and V. Vaikuntanathan, “Fully homomorphic encryption over the integers,” in *Advances in Cryptology—EUROCRYPT 2010*. Springer, 2010, pp. 24–43.
- [21] A. Viand, C. Knabenhans, and A. Hithnawi, “Verifiable fully homomorphic encryption,” arXiv preprint arXiv:2301.07041, 2023.
- [22] Z. Zhang, T. Plantard, and W. Susilo, “Reaction attack on outsourced computing with fully homomorphic encryption schemes,” in *Information Security and Cryptology-ICISC 2011*. Springer, 2012, pp. 419–436.